

Data Structures in Python

Contents

1	Introduction	4
2	What are Data Structures?	4
3	List	5
4	Mixed Data Types Inside List	5
5	Empty List	5
6	Appending Data	5
7	Index Position	6
8	Finding Index	6
9	Insert Method	6
10	Removing Items	7
10.1	Remove Method	7
10.2	Important Warning	7
11	Pop Method	7
11.1	Pop Last Item	7
11.2	Pop Specific Index	8
12	Looping Through List	8
13	Enumerate Function	8
14	Understanding enumerate()	8
15	Dynamic Participant System	9
16	Displaying Final Participants	9
17	Menu Driven Program	9
18	Upper Method	10

19 Adding Participant	10
20 Removing Participant Safely	10
21 Showing All Participants	11
22 Quitting Program	11
23 Tuple	11
24 List vs Tuple	11
25 Mutable vs Immutable	11
26 Converting List to Tuple	12
27 Tuple Example	12
28 Tuple Modification Error	12
29 Unpacking	12
30 Star Unpacking	13
30.1 Printing Values	13
31 Why Star Unpacking is Useful	13
32 Final Thoughts	13
33 Exercises	15
33.1 Exercise 1: Create List	15
33.2 Exercise 2: Add Items	15
33.3 Exercise 3: Remove Item	15
33.4 Exercise 4: Find Index	15
33.5 Exercise 5: Number Counter	15
33.6 Exercise 6: Sum of List	15
33.7 Exercise 7: Largest Number	15
33.8 Exercise 8: Tuple Practice	15
33.9 Exercise 9: Participant System	16
33.10Exercise 10: Reverse List	16
34 Challenge Questions	17
34.1 Challenge 1: Duplicate Finder	17
34.2 Challenge 2: Remove Duplicates	17
34.3 Challenge 3: Second Largest Number	17
34.4 Challenge 4: Voting System	17
34.5 Challenge 5: Password List	17
34.6 Challenge 6: Character Frequency	17
34.7 Challenge 7: Mini Shopping Cart	18
34.8 Challenge 8: Topper Finder	18

1 Introduction

Till now we stored:

- One number
- One string
- One value at a time

Example:

```
name = "Ram"  
marks = 80
```

But real programs handle lots of data.

Examples:

- List of students
- List of products
- List of messages
- List of songs

Imagine creating:

```
student1 = "Ram"  
student2 = "Hari"  
student3 = "Shyam"  
student4 = "Sita"
```

for 10,000 students.

At that point even your keyboard may resign.

That is why:

Data Structures exist.

2 What are Data Structures?

Data structures are ways to organize and store multiple data efficiently.

Python provides many data structures.

Today we learn:

- List
- Tuple

3 List

A list stores multiple values together.

Example:

```
list_of_numbers = [1, 2, 3, 4]
```

Lists are written using:

```
[]
```

4 Mixed Data Types Inside List

Lists can store different types of data together.

```
mixed_list = [1, "ram", 5.5, "Kathmandu", True]
```

Meaning:

- Integer
- String
- Float
- Boolean

all can exist together.

Python lists are very flexible.

Sometimes too flexible.

5 Empty List

```
participants = []
```

This creates empty list.

Output:

```
[]
```

Meaning:

“List exists, but currently contains nothing.”

6 Appending Data

To add new item at end of list:

```
participants.append("Ram")
```

Now list becomes:

```
["Ram"]
```

Add another:

```
participants.append("Hari")
```

Now:

```
["Ram", "Hari"]
```

7 Index Position

Every item in list has position.

Called:

Index

Example:

```
["Ram", "Hari"]
```

Positions:

```
0      1
```

Python indexing starts from:

0

Not from 1.

Python developers and mathematicians apparently dislike normal counting.

8 Finding Index

```
participants.index("Ram")
```

Output:

```
0
```

Example:

```
print("Index of Ram is", participants.index("Ram"))
```

9 Insert Method

`append()` always adds at end.

But `insert()` allows adding at specific position.

```
participants.insert(0, "Shyam")
```

Now list becomes:

```
["Shyam", "Ram", "Hari"]
```

Positions:

```
0          1          2
```

10 Removing Items

10.1 Remove Method

```
participants.remove("Hari")
```

Removes item by value.

Example:

Before:

```
["Ram", "Hari", "Shyam"]
```

After:

```
["Ram", "Shyam"]
```

10.2 Important Warning

If item does not exist:

```
participants.remove("Seeta")
```

Python gives error:

```
ValueError
```

because:

“Seeta is not in list”

Python becomes dramatic very quickly sometimes.

11 Pop Method

`pop()` removes item using index.

11.1 Pop Last Item

```
participants.pop()
```

Removes last item.

11.2 Pop Specific Index

```
participants.pop(0)
```

Removes item at index 0.

12 Looping Through List

```
participants = ["ram", "hari", "shyam"]

for name in participants:
    print(name)
```

Output:

```
ram
hari
shyam
```

13 Enumerate Function

Sometimes we need:

- Index number
- Value

both together.

Python gives:

```
enumerate()
```

Example:

```
participants = ["ram", "hari", "shyam"]

for num, name in enumerate(participants, start=1):
    print(num, name)
```

Output:

```
1 ram
2 hari
3 shyam
```

14 Understanding enumerate()

Normally indexes start from 0.

But:

```
start=1
```

tells Python:

“Start counting from 1 instead.”

Useful for showing lists to users.

Humans prefer:

```
1, 2, 3
```

instead of:

```
0, 1, 2
```

15 Dynamic Participant System

Now let users enter participant names dynamically.

```
participants = []

while True:

    new_name = input("Enter participant name (q for quit): ")

    if new_name == 'q':
        break

    participants.append(new_name)
```

16 Displaying Final Participants

```
print("List of participants")

for num, name in enumerate(participants, start=1):
    print(num, name)
```

17 Menu Driven Program

Now we create proper mini application.

```
participants = []

MENU = """
===== MENU =====
1. Add new participant
2. Remove participant
3. Show participants
Q. Quit
=====
"""
```

18 Upper Method

```
user_choice = input("Enter choice: ").upper()
```

`upper()` converts text into uppercase.

Example:

```
"q" -> "Q"
```

Useful because:

- q
- Q

both become same.

19 Adding Participant

```
if user_choice == "1":  
  
    new_name = input("Name of participant >>> ")  
  
    participants.append(new_name)
```

20 Removing Participant Safely

```
elif user_choice == "2":  
  
    remove_name = input("Name to remove >>> ")  
  
    if remove_name in participants:  
  
        participants.remove(remove_name)  
  
        print("Participant removed successfully")  
  
    else:  
        print("Participant not found")
```

Notice this:

```
if remove_name in participants
```

We first check existence before removing.
Good programmers do not trust blindly.

21 Showing All Participants

```
elif user_choice == "3":  
  
    print("List of participants")  
  
    for num, name in enumerate(participants, start=1):  
        print(num, name)
```

22 Quitting Program

```
elif user_choice == "Q":  
  
    print("Thank you for using our program")  
  
    break
```

23 Tuple

Tuple is another data structure.

Looks similar to list.

But important difference exists.

24 List vs Tuple

List:

```
["ram", "hari"]
```

Tuple:

```
("ram", "hari")
```

Notice:

- List uses []
- Tuple uses ()

25 Mutable vs Immutable

Lists are:

Mutable

Meaning:

Can be modified.

Tuples are:

Immutable

Meaning:

Cannot be modified after creation.

26 Converting List to Tuple

```
final_participants = tuple(participants)
```

Now data becomes tuple.

27 Tuple Example

```
final_participants = ("ram", "hari", "seeta")
```

Accessing values:

```
print(final_participants[1])
```

Output:

```
hari
```

28 Tuple Modification Error

```
final_participants[1] = "Shyam"
```

Python gives error.

Because tuples cannot be changed.

Once created:

“Final means final.”

29 Unpacking

Python allows unpacking values.

Example:

```
a, b, c = 1, 2, 3
```

Now:

```
a = 1  
b = 2  
c = 3
```

30 Star Unpacking

```
a, *mids, b = 1, 2, 3, 4, 5, 6, 7, 8
```

Result:

```
a = 1
mids = [2, 3, 4, 5, 6, 7]
b = 8
```

30.1 Printing Values

```
print(a)
print(mids)
print(b)
```

Output:

```
1
[2, 3, 4, 5, 6, 7]
8
```

31 Why Star Unpacking is Useful

Useful when:

- First and last values important
- Middle values grouped together

Example:

- First topper
- Middle students
- Last student

32 Final Thoughts

Today you learned:

- Lists
- Indexing
- `append()`
- `insert()`
- `remove()`

- `pop()`
- `enumerate()`
- Tuples
- Mutable vs Immutable
- Unpacking

Now your programs can store collections of data.

Congratulations.

You have officially entered real programming territory.

From now on:

Programs become much more powerful.

33 Exercises

33.1 Exercise 1: Create List

Create list of:

- 5 favorite fruits

Print all fruits using loop.

33.2 Exercise 2: Add Items

Create empty list.

Take 5 names from user and store inside list.

33.3 Exercise 3: Remove Item

Create list of cities.

Ask user which city to remove.

Remove if exists.

33.4 Exercise 4: Find Index

Store student names in list.

Ask user name and print its index.

33.5 Exercise 5: Number Counter

Store numbers inside list.

Count how many even numbers exist.

33.6 Exercise 6: Sum of List

Find total sum of numbers inside list.

33.7 Exercise 7: Largest Number

Find largest number from list manually using loop.

(No using max())

33.8 Exercise 8: Tuple Practice

Create tuple of weekdays.

Print:

- First day
- Last day

33.9 Exercise 9: Participant System

Create menu-driven participant system:

- Add participant
- Remove participant
- Show participants
- Quit

33.10 Exercise 10: Reverse List

Print list items in reverse order using loop.

34 Challenge Questions

34.1 Challenge 1: Duplicate Finder

Take 10 numbers from user.

Print:

- Duplicate numbers

Example:

Input:

```
1 2 3 2 5 1
```

Output:

```
1
2
```

34.2 Challenge 2: Remove Duplicates

Create program that removes duplicate items from list.

34.3 Challenge 3: Second Largest Number

Find second largest number from list.

Without sorting.

34.4 Challenge 4: Voting System

Create voting system:

- Users vote for candidates
- Store votes in list
- Show total votes

34.5 Challenge 5: Password List

Store list of allowed passwords.

Ask user password.

Check whether entered password exists in list.

34.6 Challenge 6: Character Frequency

Take sentence input.

Count how many times each character appears.

Example:

Input:

```
hello
```

Output:

```
h -> 1
e -> 1
l -> 2
o -> 1
```

34.7 Challenge 7: Mini Shopping Cart

Create shopping cart system:

- Add products
- Remove products
- Show cart items
- Checkout

34.8 Challenge 8: Topper Finder

Store marks of students.

Find:

- Highest marks
- Lowest marks
- Average marks

Without using built-in functions like:

```
max()
min()
sum()
```

35 Homework Advice

Lists become easy only after writing lots of programs.

Do not memorize methods blindly.

Use them.

Break them.

Experiment with them.

Because programming is not a spectator sport.